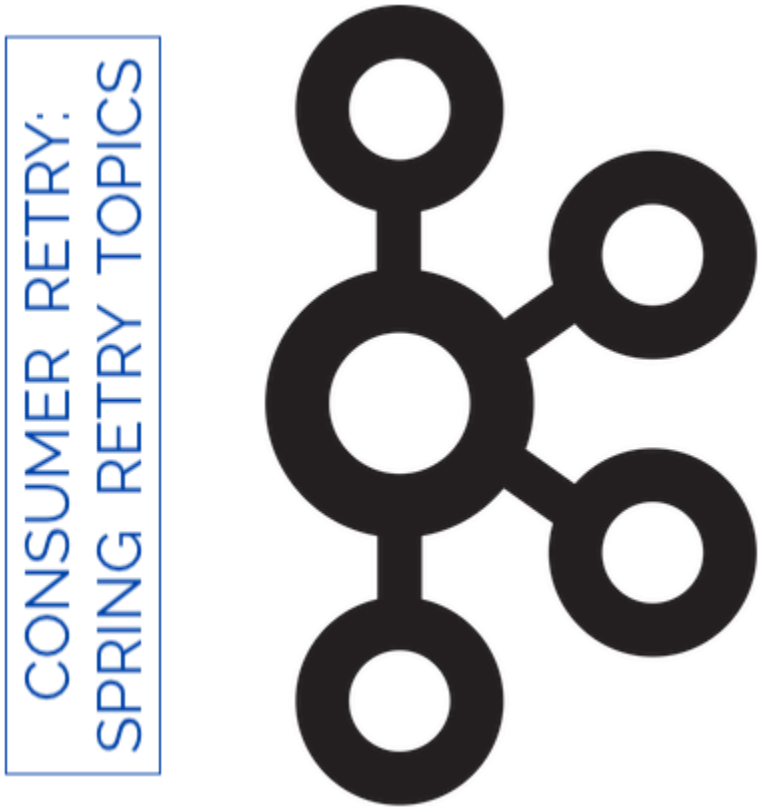
Kafka Consumer Non-Blocking Retry: Spring Retry Topics

Rob Golder - April 2023



Introduction

When an event is consumed from Kafka by an application but the application is not able to process it then it may be required to retry the event. If the retry is blocking then no other events from the same topic partition will be processed until the event being retried has completed (successfully or otherwise). A non-blocking retry pattern on the other hand ensures that those other events will still be processed while the original event is being retried. Spring Kafka provides a mechanism for retry that is non-blocking. It requires minimal code changes to implement. The trade-off with using this pattern is the loss of guaranteed event ordering.

This article describes how Spring retry topics work, and provides an accompanying Spring Boot application that demonstrates the behaviour. The source code for the application is available here.

For applications not using Spring Kafka, the second part of this article details the design and implementation of a non-blocking retry pattern that achieves the same behaviour.

Blocking vs Non-Blocking Retry

Blocking Retry

If an application is unable to process an event due to a transient problem, such as a downstream service being unavailable, then typically a blocking retry would be utilised. The reason for this is that the issue will likely affect the later events on the same topic partition. There would therefore be little value in retrying the original event in a non-blocking manner to allow these later events be processed, as they would fail in the same manner. If just the single event is being retried, once the transient issue is resolved this can complete processing before the next events are attempted.

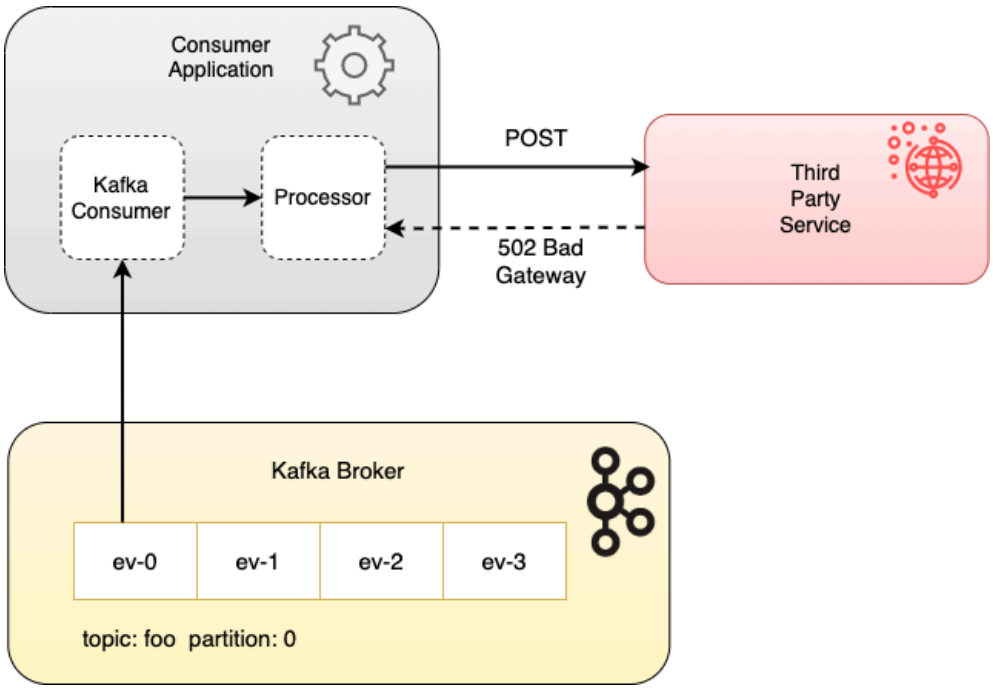


Figure 1: Using blocking retry

In this scenario the first event ev-0 is polled by the consumer. A REST call is made to a third party service which fails with a 502 Bad Gateway response. There would be little advantage in retrying ev-0 in a non-blocking manner, as when the next event is polled and processed it will likely also result in a 502 Bad Gateway from the third party service. Therefore retrying

ev-0 from the topic until this transient issue is resolved is appropriate, at which point the consumer will poll for later events ev-1 and beyond.

Blocking retry also ensures that the ordering guarantees that Kafka gives are preserved. Later events on the same topic partition can not 'jump the queue' and be processed before an earlier event when that earlier event is being retried. Maintaining event ordering can be a requirement of the system.

Spring's blocking retry is covered in the article [Kafka Consumer Retry](#).

Non-Blocking Retry

When events on a topic partition may be able to be processed even when earlier events need to be retried, then a non-blocking retry pattern ensures that a retrying event does not block the partition. An example would be when an update-item event is consumed, but the corresponding create-item event has not yet been received and processed. As the item has not yet been persisted to the database, the update cannot be applied. By retrying this update-event for a period of time means that the create-item event can be received and processed subsequent to the update-item. When the original update-item event is next retried, it can be successfully applied to the item entity that has since been created.

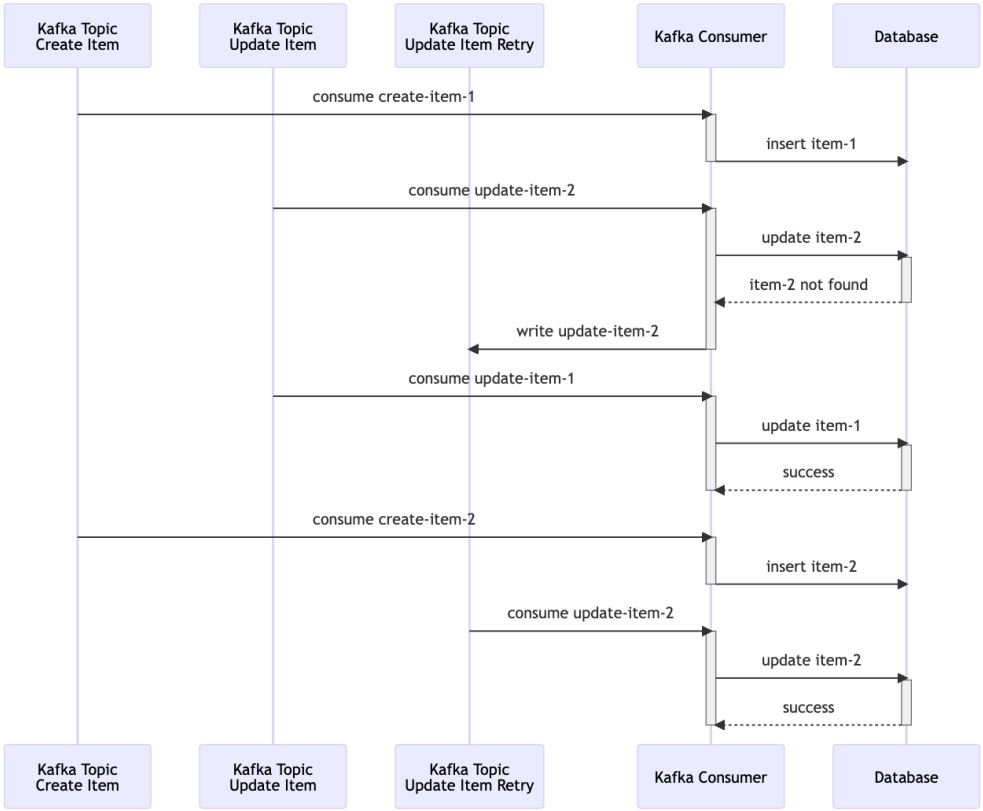


Figure 2: Non-blocking retry flow

With this non-blocking retry pattern, while the update-item event is being retried, the other events on the same topic partition are still being consumed and processed. So long as their corresponding create-item events have been processed, they can be successfully applied. There is no guarantee then on the order that these update-item events will be applied, given that an earlier event may be being retried while a later event is successfully processed.

Spring Retryable Topics

Spring uses retryable topics to achieve non-blocking retry. Rather than retry an event from the original topic in a blocking manner, Spring Kafka instead writes the event to a separate retry topic. The event is marked as consumed from the original topic, so the next events continue to be polled and processed. Meanwhile a separate instance of the same consumer is instantiated by Spring as the consumer for the retry topic. This ensures that a single consumer instance is not polling and receiving events from both the original and a retry topic.

If the event needs to be retried multiple times, it can either be retried from the single retry topic, or it can be written to a further retry topic. The advantage of a single retry topic is that there are less topics to be dealing with. The downside is that events being retried from it will be blocking this retry topic. Alternatively any number of further retry topics can be used, ensuring each is not blocked when the event is retried. Each retry topic might have a longer back-off reflecting the need to give the system more time to be in a state that it can process the event successfully.

Once all retries are exhausted a dead letter topic can be configured to write the event to. Optionally a method in the consumer class can be annotated to consume from this topic.

Retryable Topic Configuration

The Kafka consumer is configured to use retryable topics by annotating the consumer with @RetryableTopic. The following are the main configuration options available for this annotation:

Configuration	Usage
attempts	The number of attempts before the event is sent to the dead letter topic.

backoff	The backoff to apply to each retry. This is used to determine the delay between retries.
timeout	The maximum time to retry before the event is sent to the dead letter topic.
fixedDelayTopicStrategy	Whether to use a single or multiple retry topics.
topicSuffixingStrategy	Whether to suffix the topic with an incrementing index or the delay value.
dltStrategy	Whether to create a dead letter topic, and how to handle delivery failures.
include / includeNames	The exception types that should be retried.
exclude / excludeNames	The exception types that should not be retried.
traversingCauses	Whether to traverse the exception chain to look for included or excluded exception types.
autoCreateTopics	Whether the retry and dead letter topics should be automatically or manually created.
replicationFactor	The replication factor to apply for automatically created topics.
numPartitions	The number of partitions to apply for automatically created topics.

Spring Boot Demo

The accompanying Spring Boot project demonstrates using Spring Kafka retryable topics. It covers the use case where an update-item event is received before its corresponding create-item event, so the item is not available in the database to be updated. The update-item is therefore retried until the item has been created.

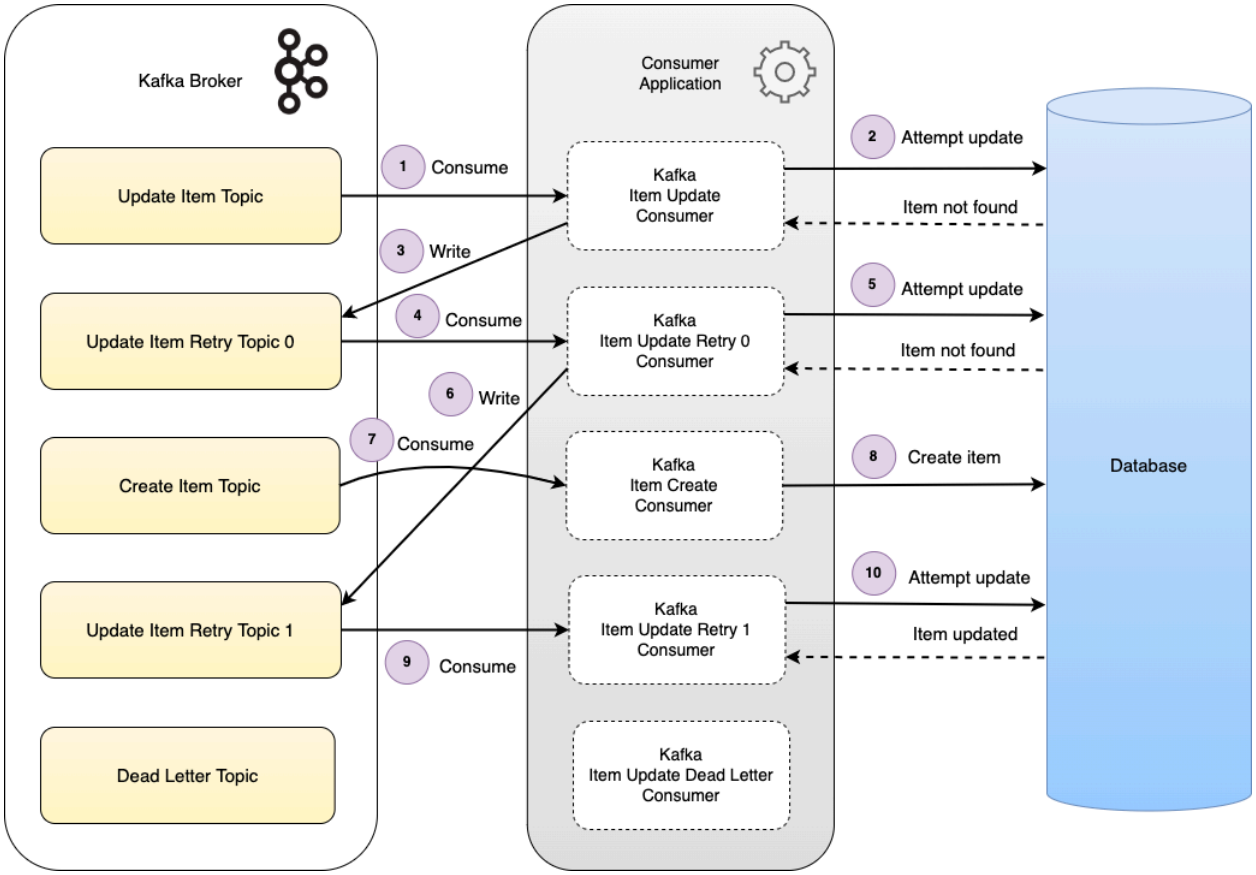


Figure 3: Non-blocking retry with Spring Kafka

The diagram illustrates multiple retry topics in use. The update-item event is consumed and retried multiple times. At steps (2) and (5) the item is not found when the update is attempted, so the event is written to the next retry topic. The item is created via the create-item event at step (8). On the next retry at step (10) the update is successfully applied. In this case as the update is successful no more retries occur, and the dead letter topic is not written to.

The main class of interest is the UpdateItemConsumer. The constructor is autowired with the retry configuration arguments defined in the application.yml properties file. The listener method is annotated with both the @KafkaListener annotation to mark it as the consumer for the specified topic update-item, as well as the @RetryableTopic annotation.

Finally a dead letter topic handler method is annotated with @DltHandler which simply logs any event received from the dead letter topic, which occurs once all retries for an event are exhausted.

```
@Slf4j
@Component
public class UpdateItemConsumer {

    private final ItemService itemService;

    @RetryableTopic(
            attempts = "#{'${demo.retry.maxRetryAttempts}'}",
            autoCreateTopics = "#{'${demo.retry.autoCreateRetryTopics}'}",
            backoff = @Backoff(delayExpression =
"#{'${demo.retry.retryIntervalMilliseconds}'}", multiplierExpression =
"#{'${demo.retry.retryBackoffMultiplier}'}"),
            fixedDelayTopicStrategy = FixedDelayStrategy.MULTIPLE_TOPICS,
            include = {RetryableMessagingException.class},
            timeout = "#{'${demo.retry.maxRetryDurationMilliseconds}'}",
            topicSuffixingStrategy =
TopicSuffixingStrategy.SUFFIX_WITH_INDEX_VALUE)
    @KafkaListener(topics = "#{'${demo.topics.itemUpdateTopic}'}",
containerFactory = "kafkaListenerContainerFactory")
    public void listen(@Payload final String payload) {
       log.info("Update Item Consumer: Received message with payload: " +
payload);
       try {
           UpdateItem event = JsonMapper.readFromJson(payload,
UpdateItem.class);
           itemService.updateItem(event);
       } catch (RetryableMessagingException e) {
           // Ensure the message is retried.
           throw e;
       } catch (Exception e) {
           log.error("Update item - error processing message: " +
e.getMessage());
       }
   }

   @DltHandler
   public void dlt(String data, @Header(KafkaHeaders.RECEIVED_TOPIC)
String topic) {
```

```
        log.error("Event from topic "+topic+" is dead lettered - event:" +
data);
    }
}
```

The @RetryableTopic is configured to use multiple retry topics, ensuring each retry is non-blocking. The majority of the configuration such as the backoff and retry timeout can be changed in the application.yml, or overridden in the application-test.yml for the Spring Boot integration tests. This ensures that the behaviour can be changed without requiring a code change or rebuild.

Running The Demo

To run the demo docker is used to bring up a dockerised Kafka and Zookeeper for the application to integrate with, in the provided docker-compose.yml. The Conduktor Platform is also started in docker. Conduktor is a web application that provides a UI over the broker and topic, which is useful to see what is happening under the hood. The application is built and run as an executable jar file.

```
docker-compose up -d
mvn clean install
```

```
java -jar target/kafka-spring-retry-topics-1.0.0.jar
```

The Kafka command line tools are used to produce the necessary events to the broker, from which the application will consume. First the update-item event is written, which attempts to update the associated item's status to ACTIVE. This event can be observed being written to each of the retry topics while it is unable to find the item in the database.

```
kafka-console-producer \
--topic update-item \
```

```
--broker-list kafka:29092
```

```
{"id": "b346d83e-f2db-4427-947d-3e239111d6db", "status": "ACTIVE"}
```

The application provides a REST endpoint (in ItemController) that can be hit to retrieve the item status. curl can be used to check the status of the item. As the item is not found at this point, a 404 Not Found is returned:

```
curl -X GET
```

```
http://localhost:9001/v1/demo/items/b346d83e-f2db-4427-947d-3e239111d6db/s
```

```
tatus
```

Next the create-item event is written to Kafka:

```
kafka-console-producer \
--topic create-item \
--broker-list kafka:29092
```

```
{"id": "b346d83e-f2db-4427-947d-3e239111d6db", "name": "my-new-item"}
```

The event is consumed by the application, and the item is created in the database with status PENDING. If the update-item is still being retried then the status update will be applied. Using curl the status of the item will now be returned.

The Conduktor web application can be used to view the retry topics. Log in to Conduktor at:

`http://localhost:8080`

Use credentials:

`admin@conduktor.io` / `admin`.

Navigate to the Console, and view the hidden topics:

Name	Tags	Partitions	Replication	Count	
__consumer_offsets		50	1	9	···
create-item		1	1	0	···
update-item		1	1	1	···
update-item-dlt		1	1	0	···
update-item-retry-0		1	1	1	···
update-item-retry-1		1	1	1	···
update-item-retry-2		1	1	0	···

Figure 4: Retry and dead letter topics

Here we see the retry topics and dead letter topic (with dlt suffix) that have been created. In this case a single update-item has been retried via both the update-item-retry-0 and update-item-retry-1 topics.

Navigating to the consumer groups tab in Conduktor also highlights the fact that a separate instance of the consumer has been created for each retry topic, as well as the dead letter topic.

The steps for running the demo are also covered in the project Readme.

Spring Boot Integration Tests

Integration testing the application is an essential part of its development, and ensuring the correctness of the retry behaviour is one aspect that must be thoroughly tested. Spring Kafka provides an in-memory embedded Kafka broker for use in the Spring Boot tests. This enables the developer to test and prove that the application is able to connect to and consume from the broker. It also means that the retry behaviour can be tested, as events are written to the retry topics by the Spring Kafka library, and consumed by the application's retry consumer instances.

The accompanying Spring Boot project defines the retryable integration tests in KafkaIntegrationTest. Along with the @SpringBootTest annotation that marks the test as an integration test that brings up the Spring context, the class is annotated with the @EmbeddedBroker annotation. This is all that is required to bring up the embedded broker at test time.

The class defines three tests. First the happy path scenario where a number of items are first created via create-item events before the corresponding update-item events are sent to the broker. Next the scenario where the update-item events are received first, and hence are retried until the create-item events are received. The third scenario is where the create-item events are never sent, so the update-item events are retried until retries are exhausted, and then written to the dead letter topic.

Conclusion

Spring Kafka makes it straightforward to implement a non-blocking retry pattern for an application consumer. Simpy with the addition of an annotation on the listener method this retry behaviour is enabled. The retry functionality is executed by the Spring Kafka library under the hood, leaving nothing further for the developer to implement. The retry is fully configurable with features such as back-off and retry timeout. The trade-off with this non-blocking retry pattern is that the order that events are processed in can no longer be guaranteed, as a later event may complete processing while an earlier event is being retried.

Source Code

The source code for the accompanying Spring Boot demo application is available here:

https://github.com/lydtechconsulting/kafka-spring-retry-topics/tree/v1.0.0

More On Kafka Consumer Retry

- Kafka Consumer Non-Blocking Retry Pattern: the second article on non-blocking retry, detailing a pattern that can be applied when not using Spring Kafka.
- Kafka Consumer Retry: provides an overview on consumer retry, the factors to consider when choosing a retry pattern, and details stateless and stateful blocking retry patterns.